

Praktikum 3 - Computer Vision

**Diajukan untuk memenuhi praktikum 3 Mata Kuliah II4012 AI for Business pada Semester
genap tahun Akademik 2025/2026**

Oleh

Raditya Zaki

18223086

Mochamad Ikhbar A

18223050



**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026**

DAFTAR ISI

DAFTAR ISI.....	2
DAFTAR TABEL.....	4
DAFTAR GAMBAR	5
BAB I	8
PENDAHULUAN.....	8
1.1. Latar Belakang	8
1.2. Rumusan Masalah	8
1.3. Tujuan Praktikum	8
BAB II.....	9
LANDASAN TEORI.....	9
2.1. Image Classification & CNN	9
2.2. Transfer Learning	9
2.3. EfficientNetB2	10
2.4. Batch Normalization.....	10
2.5. Global Average Pooling (GAP)	10
2.6. Metrik Evaluasi	10
BAB III	12
METODOLOGI DAN EKSPERIMEN	12
3.1. Deskripsi Dataset.....	12
3.2. Preprocessing	13
3.2.1. Standarisasi Ukuran.....	13
3.2.2. Augmentasi Data.....	14
3.2.3. Konversi Tensor dan Normalisasi Statistik.....	14
3.3. Skenario Model 1 (CNN Base)	15
3.4. Skenario Model 2 (Pre-trained).....	17
BAB IV	19
HASIL DAN PEMBAHASAN	19
4.1. Analisis Proses Training.....	19
4.1.1. CNN Model.....	19
4.1.2. EfficientNetB2	22
4.2. Perbandingan Metrik Performa	25
4.3. Analisis Confusion Matrix	27
4.3.1. CNN Model.....	28
4.3.2. EfficientNetB2	31
4.4. Pembahasan.....	33
BAB V.....	35
PENUTUP.....	35
5.1. Kesimpulan	35
5.2. Saran.....	35

DAFTAR TABEL

Tabel 4.1 Environment Pengujian	17
Tabel 4.2 Konfigurasi Hyperparameter Pelatihan Custom CNN	17
Tabel 4.3 Ringkasan Metrik Performa Terbaik Custom CNN (Epoch 64)	19
Tabel 4.4 Konfigurasi Hyperparameter Pelatihan EfficientNetB2	20
Tabel 4.5 Ringkasan Metrik Performa Terbaik EfficientNetB2 (Epoch 56)	22
Tabel 4.6 Komparasi Metrik Performa: Custom CNN vs EfficientNetB2	23
Tabel 4.7 Pasangan Kelas dengan Kesalahan Prediksi Tertinggi (Custom CNN)	27
Tabel 4.8 Sepuluh Kelas dengan Performa Terburuk (Custom CNN)	27
Tabel 4.9 Pasangan Kelas dengan Kesalahan Prediksi Tertinggi (EfficientNetB2)	30
Tabel 4.10 Sepuluh Kelas dengan Performa Terburuk (EfficientNetB2)	30

DAFTAR GAMBAR

Gambar 3.1 Random Training Sample Image.....	9
Gambar 3.2 Distribusi Kelas Pada Setiap Set	10
Gambar 3.3 Hasil Augmentasi	12
Gambar 3.4 Arsitektur Model CNN	13
Gambar 3.5 Arsitektur Model EfficientNetB2	15
Gambar 4.1 Grafik Riwayat Pelatihan (Loss & Accuracy) Custom CNN.....	19
Gambar 4.2 Grafik Riwayat Pelatihan (Loss & Accuracy) EfficientNetB2	22
Gambar 4.3 Confusion Matrix Custom CNN pada Test Set	25
Gambar 4.4 Confusion Matrix EfficientNetB2 pada Test Set	28

BAB I

PENDAHULUAN

1.1. Latar Belakang

Implementasi *computer vision* pada pesawat terbang merupakan salah satu bentuk masalah *Fine Grained Visual Categorization* (FGVC). Berbeda dengan klasifikasi objek umum, FGVC memiliki tingkat kesulitan yang lebih tinggi karena tingginya tingkat kemiripan visual antar kelas. Secara tradisional, pemrosesan gambar seperti ini diselesaikan dengan merancang dan melatih arsitektur *Convolutional Neural Network* (CNN). Pendekatan base cnn model ini memberikan kebebasan dalam mendesain arsitektur, namun sering kali dihadapkan pada kendala seperti kebutuhan komputasi yang besar, waktu pelatihan yang lama, serta kerentanan terhadap overfitting jika jumlah data terbatas.

Pada praktikum kali ini, kami akan melakukan eksperimentasi penggunaan *cnn base model* dan *pre-trained model* untuk kasus ini dan membuktikan mana yang lebih efektif dalam memprediksi, efisien dalam komputasi, dan model mana yang berhasil menggeneralisasi dataset.

1.2. Rumusan Masalah

1. Bagaimana cara merancang dan melatih arsitektur *base CNN model* dari nol untuk mengenali fitur spesifik pada gambar pesawat terbang?
2. Bagaimana cara mengimplementasikan *pre-trained model EfficientNetB2* menggunakan teknik *fine-tuning* pada dataset FGVC-Aircraft?
3. Bagaimana perbandingan performa antara base cnn model dan pretrained model dalam hal metrik akurasi, kecepatan konvergensi, dan efisiensi waktu komputasi?

1.3. Tujuan Praktikum

1. Merancang dan mengevaluasi jaringan *Convolutional neural Network* dasar pada dataset berukuran kecil.
2. Menerapkan teknik *transfer learning* dengan melakukan penyesuaian lapisan klasifikasi pada arsitektur EfficientNetB2.
3. Membandingkan secara langsung hasil prediksi kedua model untuk memberikan justifikasi komputasional dan bisnis terkait arsitektur mana yang paling layak digunakan dalam kasus *fine grained visual categorization*.

BAB II

LANDASAN TEORI

2.1. Image Classification & CNN

Image classification adalah salah satu cabang dari *computer vision* yang bertujuan untuk mengenali dan memberikan label kategori pada sebuah gambar digital berdasarkan fitur visual yang terkandung di dalamnya. Teknik yang paling umum dan terbukti sangat efektif untuk menangani tugas ini adalah *Convolutional Neural network*.

Arsitektur CNN didesain khusus untuk memproses data dua dimensi seperti gambar dengan mempertahankan struktur spasialnya. Jaringan CNN secara umum terdiri dari tiga komponen atau lapisan utama

- ***Convolutional Layer***

Merupakan inti dari CNN yang bertugas mengekstraksi fitur dari input. Lapisan ini mengoperasikan sebuah filter (atau kernel) yang bergeser di seluruh permukaan piksel atau mendeteksi berbagai pola, mulai dari pola sederhana seperti garis dan tepi, hingga pola kompleks seperti bentuk bagian pesawat.

- ***Pooling Layer***

Bertugas untuk melakukan reduksi dimensi dari peta fitur yang dihasilkan oleh *convolutional layer*. Proses ini bertujuan untuk mengurangi beban komputasi dan meminimalisasi *overfitting*, sekaligus mempertahankan informasi fitur yang paling dominan

- ***Fully Connected Layer***

Lapisan akhir pada CNN dimana data yang telah diekstraksi dan direduksi dimensinya diubah menjadi array satu dimensi (*flattening*). FCL berfungsi untuk melakukan klasifikasi akhir dengan memberikan probabilitas gambar masuk ke dalam kelas kelas tertentu (pada kasus ini 20 kelas)

2.2. Transfer Learning

Transfer Learning adalah sebuah metode *machine learning* di mana pengetahuan atau pola fitur yang telah dipelajari oleh model pada suatu *source task* digunakan kembali untuk menyelesaikan tugas lain yang relevan. Metode sangat efektif terutama ketika melatih model *deep learning* dari nol menjadi tidak efisien atau saat ketersediaan data target yang terbatas. Model *pre-trained* seperti **EfficientNet** umumnya telah dilatih pada dataset yang sangat besar seperti **ImageNet** (berisi lebih dari 14 juta gambar dengan 1000 kategori). Melalui proses pelatihan ini, model telah “belajar” dan menguasai hirarki visual yang solid seperti cara membedakan warna, gradien, bentuk melingkar, hingga fitur kompleks. Oleh karena itu, ketika digunakan untuk tugas baru, model hanya perlu

melakukan sedikit *fine tuning* atau penyesuaian untuk memahami fitur spesifik kelas tersebut sehingga konvergensi tercapai lebih cepat dan akurasi meningkat tajam.

2.3. EfficientNetB2

EfficientNet adalah keluarga arsitektur CNN yang diperkenalkan oleh periset di Google. Berbeda dengan pendekatan tradisional yang memperbesar jaringan hanya pada satu dimensi (seperti membuat layer semakin dalam/banyak), EfficientNet memperkenalkan konsep *Compound Scaling*.

Keunggulan utama EfficientNet (khususnya varian B2) adalah arsitekturnya diskalakan secara matematis dan merata pada tiga dimensi utama secara bersamaan:

- **Depth**
Penambahan jumlah lapisan agar model mampu menangkap fitur yang lebih rumit
- **Width**
Penambahan jumlah *channels* pada setiap layer untuk menangkap detail fitur yang lebih banyak pada suatu tingkat resolusi
- **Resolution**
Peningkatan dimensi *input image*, di mana EfficientNetB2 secara spesifik dioptimalkan untuk menerima input 288x288 piksel. Metode *compound scaling* memastikan EfficientNetB2 memiliki efisiensi parameter yang luar biasa, mencapai akurasi yang lebih tinggi dengan beban komputasi (FLOPs) yang jauh lebih ringan dibandingkan dengan arsitektur klasik seperti ResNet atau VGG.

2.4. Batch Normalization

Merupakan teknik untuk menstabilkan dan mempercepat proses pelatihan (*training*) pada deep neural network. Lapisan ini melakukan normalisasi terhadap *output* dari *layer* sebelumnya (membuat rata-rata mendekati 0 dan variansi mendekati 1) di setiap *mini-batch*. Teknik ini sangat krusial pada model yang dilatih dari awal (*from scratch*) untuk mencegah *vanishing/exploding gradients*.

2.5. Global Average Pooling (GAP)

Merupakan alternatif dari operasi *flattening* tradisional. GAP bekerja dengan cara mengambil nilai rata-rata dari keseluruhan *feature map* pada *channel* terakhir, sehingga mereduksi matriks spasial menjadi sebuah vektor satu dimensi. Selain menurunkan jumlah parameter secara drastis (yang berdampak pada berkurangnya risiko *overfitting*), GAP juga membuat model lebih tangguh terhadap pergeseran posisi objek (*spatial translation*) di dalam gambar.

2.6. Metrik Evaluasi

Untuk mengukur seberapa baik kinerja model CNN dasar dan pre-trained model, diperlukan metrik evaluasi yang dihitung melalui parameter True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN) dari Confusion Matrix:

- **Accuracy (Akurasi):** Rasio prediksi benar (baik positif maupun negatif) dibandingkan dengan keseluruhan data yang diuji. Akurasi efektif digunakan jika pembagian jumlah data antar kelas seimbang (balanced dataset).
- **Precision (Presisi):** Mengukur ketepatan model dalam memprediksi kelas positif. Artinya, dari semua pesawat yang ditebak model sebagai 'Airbus A320', berapa persentase yang benar-benar merupakan Airbus A320.
- **Recall (Sensitivitas):** Mengukur kelengkapan (completeness) prediksi model. Artinya, dari total keseluruhan gambar 'Airbus A320' yang ada di dataset, berapa persentase yang berhasil dideteksi dengan benar oleh model.
- **F1-Score:** Nilai rata-rata harmonik (harmonic mean) dari Precision dan Recall. F1-Score merupakan metrik evaluasi yang sangat penting untuk studi kasus ini karena memberikan gambaran keandalan model secara menyeluruh, terutama jika ada varian pesawat yang memiliki jumlah sampel data yang tidak seimbang (imbalanced).

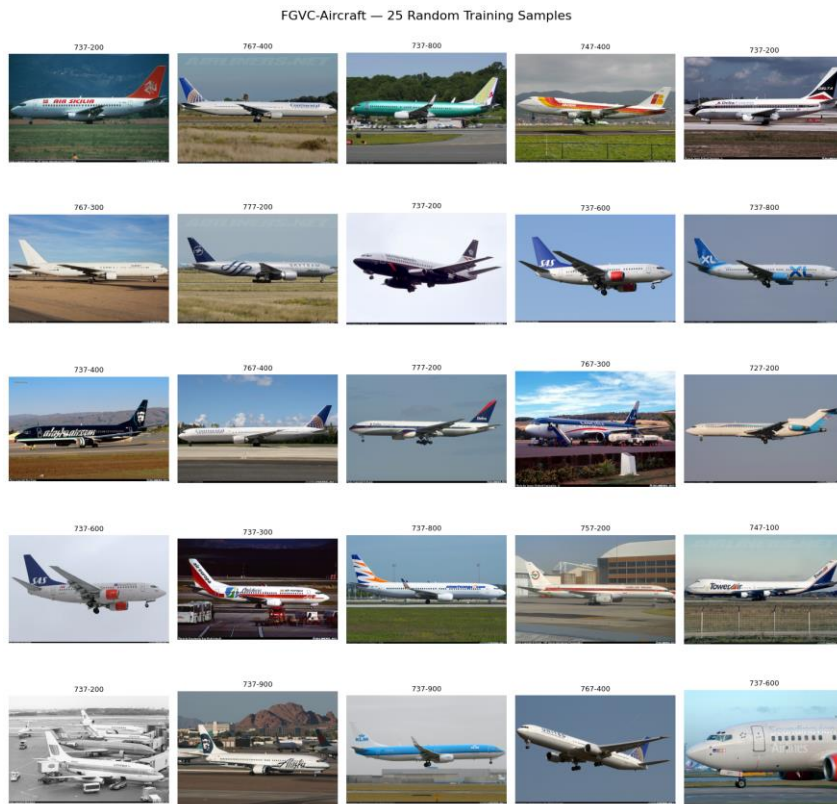
BAB III

METODOLOGI DAN EKSPERIMEN

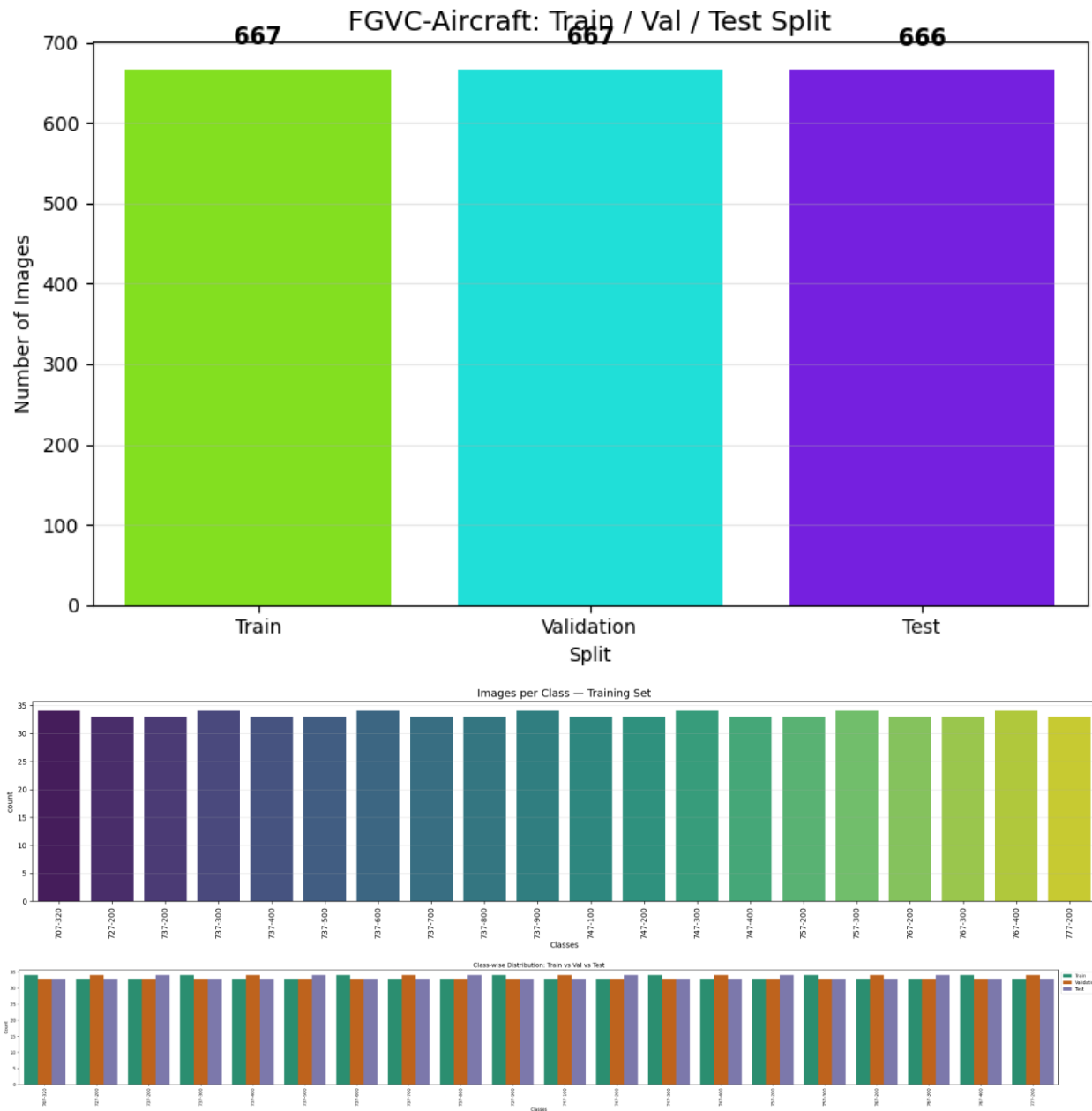
3.1. Deskripsi Dataset

Dataset yang digunakan dalam penelitian ini merupakan bagian dari dataset *Fine Grained Visual Categorization Aircraft*. Dataset ini dipilah menjadi 20 kelas varian pesawat terbang. Untuk menjaga objektivitas evaluasi dan meminimalisasi *data leakage*, dataset dibagi menjadi tiga kelompok yaitu train.csv, val.csv, dan test.csv dengan distribusi sebagai berikut

- Data Latih (Train Data): Terdiri dari 667 gambar, digunakan sebagai bahan pembelajaran utama bagi model untuk mengenali pola fitur pesawat.
- Data Validasi (Validation Data): Terdiri dari 667 gambar, digunakan secara bersamaan saat proses training berlangsung untuk mengukur tingkat konvergensi dan mencegah overfitting (sebagai acuan untuk metrik Early Stopping).
- Data Uji (Test Data): Terdiri dari 666 gambar, digunakan secara eksklusif pada tahap akhir untuk mengevaluasi performa pengenalan klasifikasi unseen data (data yang tidak pernah dilihat model sebelumnya).



Gambar 3.1 Random Training Sample Image



Gambar 3.2 Distribusi Kelas Pada Setiap Set

Dari ketiga histogram diatas, terlihat bahwa kelas pada masing masing set sudah terdistribusi secara merata, data sudah cukup seimbang sehingga tidak perlu adanya *imbalance handling*.

3.2. Preprocessing

3.2.1. Standarisasi Ukuran

Untuk memastikan model menerima *tensor* masukan dengan ukuran spasial yang seragam, semua gambar mengalami perubahan ukuran. Dimensi akhir yang digunakan untuk eksperimen ini adalah 288x288 piksel dengan 3 saluran warna (RGB).

- Fase Validasi

- Gambar langsung diubah ukurannya secara presisi menjadi resolusi akhir 288x288 piksel
- Fase Pelatihan
Pendekatan dua tahap digunakan untuk tujuan augmentasi spasial. Gambar pertama tama diperbesar menjadi resolusi 320x320, lalu dilakukan *cropping* secara acak untuk mendapatkan ukuran akhir 288x288 px. Pendekatan ini memastikan model tidak hanya menghafal posisi pesawat yang selalu berada tepat di tengah

3.2.2. Augmentasi Data

Data augmentasi diaplikasikan pada dataset pelatihan secara dinamis saat data dimuat (on-the-fly). Berikut adalah operasi augmentasi yang diaplikasikan ke dalam model:

- Random Crop: Gambar beresolusi 320x320 dipotong secara acak menjadi matriks berukuran 288x288. Hal ini melatih model untuk mengenali fitur pesawat (seperti sayap, ekor, dan moncong) yang posisinya bergeser dari bingkai asal.
- Random Horizontal Flip: Terdapat probabilitas sebesar 50% ($p=0.5$) di mana gambar akan dicerminkan secara horizontal. Orientasihadap pesawat ke kiri maupun ke kanan tidak merubah klasifikasi jenis, sehingga transformasi ini efektif menggandakan persepsi variasi arah sudut pandang.
- Color Jitter: Karena dataset gambar bersumber dari foto-foto yang diambil dalam kondisi pencahayaan, cuaca, dan kamera yang berbeda-beda, transformasi Color Jitter diaplikasikan untuk menambah ketahanan model terhadap distorsi spektrum. Nilai brightness (kecerahan), contrast (kontras), dan saturation (saturasi) divariasikan hingga 20% (0.2), sementara nilai hue (rona warna) divariasikan hingga 5% (0.05).
- Random Rotation: Gambar diputar dengan derajat kemiringan acak dalam rentang rotasi hingga 10 derajat. Pesawat dalam gambar aslinya sering kali ditangkap ketika bermanuver, lepas landas, atau mendarat dengan sudut kemiringan bervariasi. Rentang derajat yang tidak terlalu besar digunakan untuk menghindari pemotongan (kliping) area piksel yang signifikan.

3.2.3. Konversi Tensor dan Normalisasi Statistik

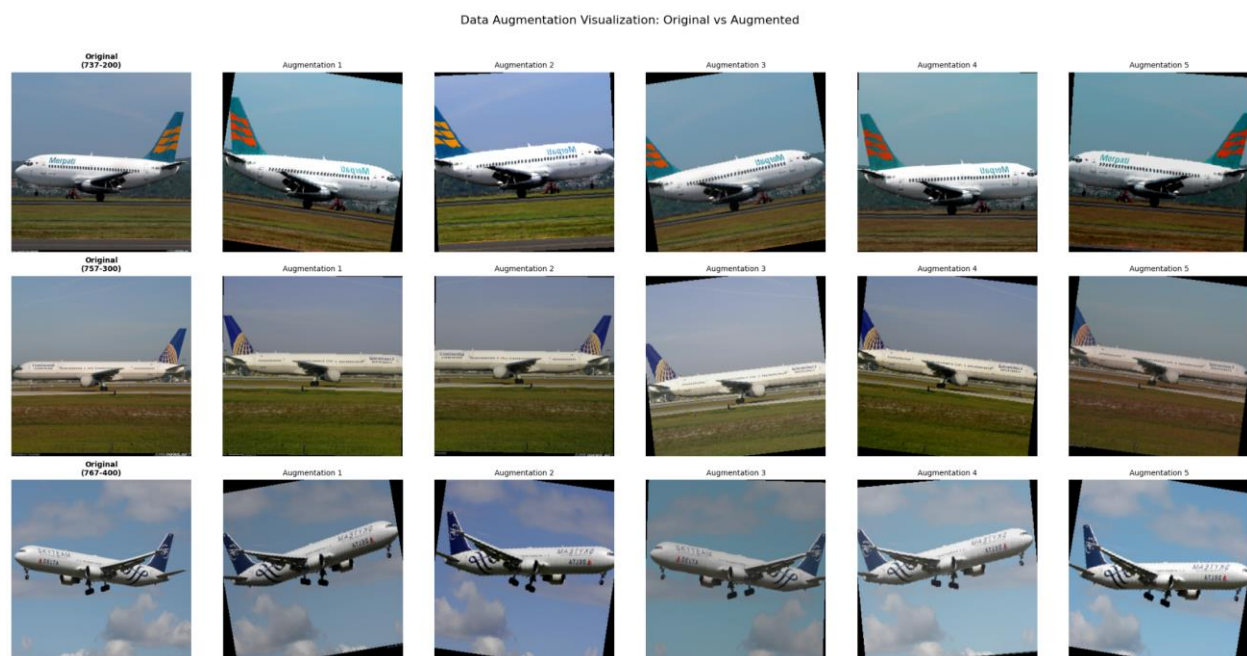
Setelah manipulasi ukuran dan augmentasi spasial-visual selesai, representasi piksel diubah menjadi PyTorch Tensors, yang merubah skala intensitas warna standar bernilai 0 hingga 255 menjadi rentang float 0.0 hingga 1.0.

Langkah terakhir dari sistem *preprocessing* ini adalah normalisasi statistik (Z-score normalization), yang menstandarisasi rentang distribusi nilai piksel menjadi terpusat pada nilai rata-rata (mean) 0 dan standar deviasi 1. Normalisasi dilakukan dengan menggunakan nilai mean dan standard deviation (std) turunan dari dataset ImageNet, mengingat EfficientNetB2 dalam penelitian ini diinisialisasi dengan bobot pre-trained (Transfer Learning). Nilai konstanta yang digunakan secara seragam di seluruh saluran gambar (R, G, B) adalah:

Mean: [0.485, 0.456, 0.406]

Std: [0.229, 0.224, 0.225]

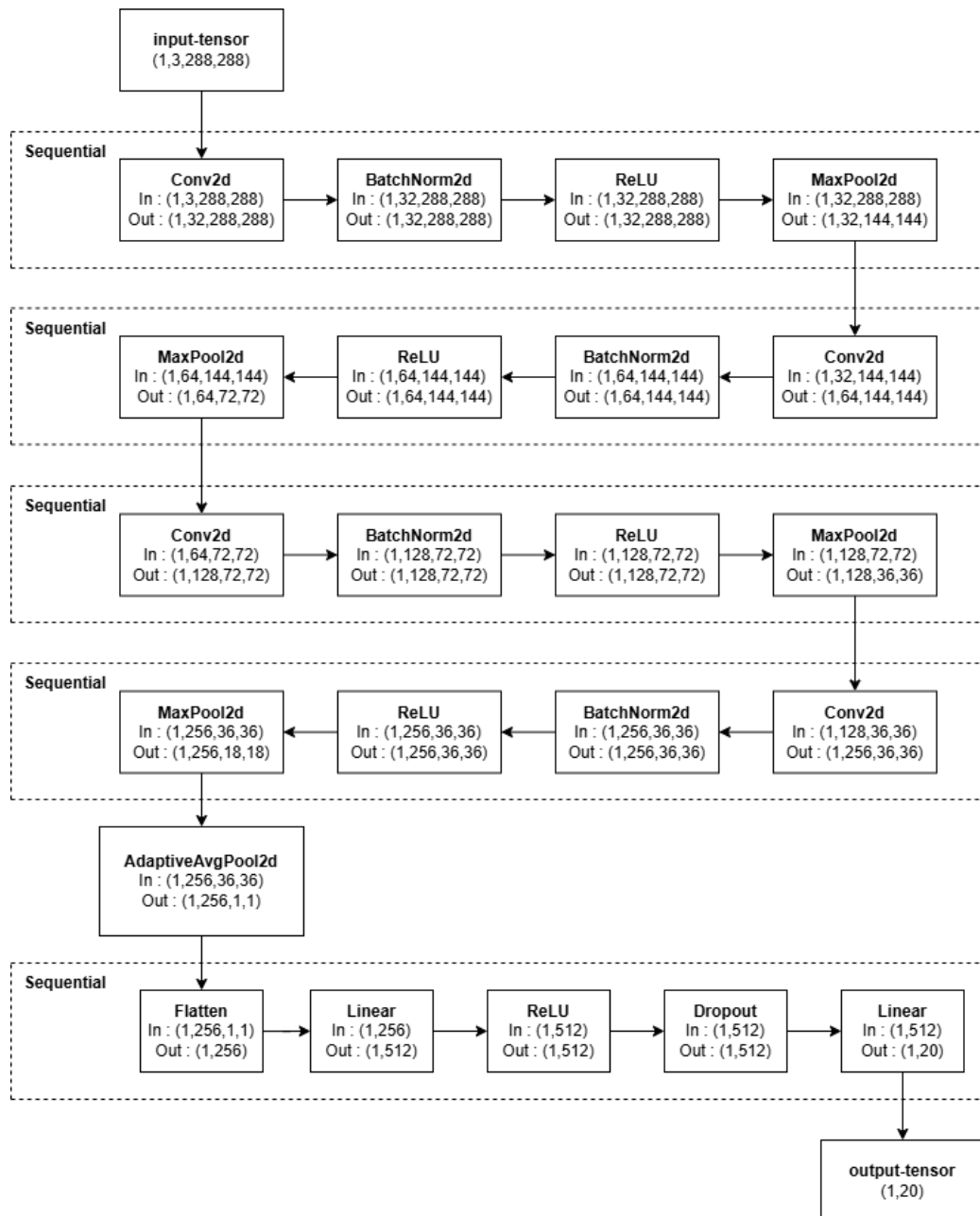
Meskipun model Custom CNN dilatih dari awal (from scratch), normalisasi yang sama tetap digunakan untuk memastikan perbandingan performa (apple-to-apple comparison) yang stabil antara arsitektur dasar dan arsitektur pre-trained. Pipeline diatur agar dapat dengan efisien menyiapkan komputasi turunan (gradients) secara seimbang di seluruh lapisan jaringan saraf, sehingga mencegah fenomena *vanishing* atau *exploding gradients* pada langkah-langkah propagasi konvolusional.



Gambar 3.3 Hasil Augmentasi

3.3. Skenario Model 1 (CNN Base)

Pada skenario model pertama ini, arsitektur *Convolutional Neural Network* dibangun sepenuhnya dari awal (*from scratch*) menggunakan modul `nn.Module` dari library PyTorch, tanpa memanfaatkan bobot konvolusi pretrained dari dataset manapun. Seluruh parameter diinisialisasi secara acak dan dipelajari eksklusif dari dataset FGVC-Aircraft. Strategi pelatihan yang digunakan adalah ***Train from Scratch***, di mana tidak ada satu pun layer yang dikunci gradiennya.



Gambar 3.4 Arsitektur Model CNN

Arsitektur CNN terdiri dari empat *convolutional block* yang tersusun secara sistematis, masing-masing dirancang untuk mengekstraksi representasi fitur pada tingkat abstraksi yang berbeda:

- **Convolutional Block (1–4)**

Setiap block terdiri dari tiga lapisan berurutan: lapisan Conv2d dengan kernel 3×3 dan padding 1 untuk mempertahankan dimensi spasial, lapisan BatchNorm2d untuk menstabilkan distribusi aktivasi sehingga proses *training from scratch* lebih stabil dan konvergen lebih cepat, serta lapisan MaxPool2d dengan stride 2 yang mereduksi resolusi

spasial secara bertahap ($288 \times 288 \rightarrow 144 \times 144 \rightarrow 72 \times 72 \rightarrow 36 \times 36 \rightarrow 18 \times 18$) sambil meningkatkan kedalaman channel ($3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$).

- **Global Average Pooling (GAP)**

Setelah block ke-4, lapisan `AdaptiveAvgPool2d(1)` merata-ratakan setiap channel feature map menjadi satu nilai skalar, menghasilkan vektor fitur berukuran 256 elemen. Penggunaan GAP menggantikan operasi *flatten* langsung yang akan menghasilkan vektor 82.944 elemen ($256 \times 18 \times 18$), sehingga jumlah parameter pada *classifier head* dapat ditekan secara drastis dari ~42 juta menjadi ~531 ribu parameter secara keseluruhan.

- **Dropout Layer**

Diterapkan dengan probabilitas penonaktifan 0,5 untuk mereduksi ko-adaptasi antar neuron dan mencegah *overfitting* yang rentan terjadi pada model yang dilatih dari nol dengan data terbatas.

- **Linear Layer**

Pemetaan *fully connected* yang mendistribusikan 512 representasi fitur menuju 20 *output features* yang merepresentasikan probabilitas prediksi terhadap 20 kelas pesawat.

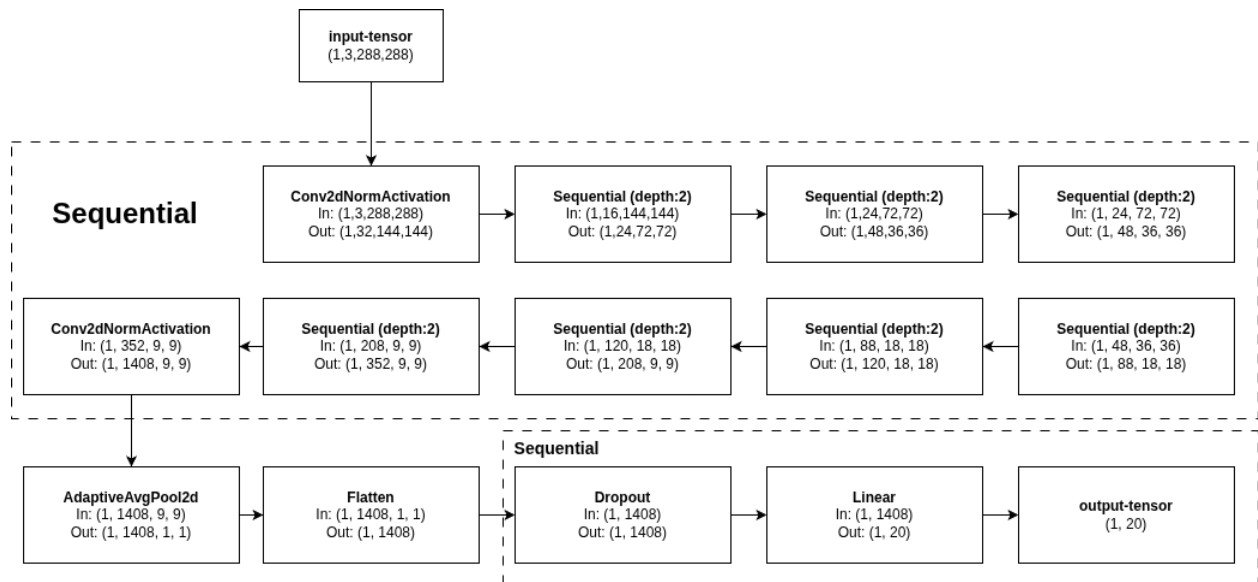
Setiap keputusan desain pada arsitektur CNN ini didasarkan pada pertimbangan keseimbangan antara kapasitas representasi model dan risiko *overfitting* mengingat keterbatasan jumlah train set.

- **4 Convolutional Block** dipilih sebagai titik tengah antara kapasitas yang terlalu dangkal (*underfitting*) dan terlalu dalam (*overfitting*). Penambahan block lebih lanjut tidak sebanding dengan jumlah data yang tersedia.
- **Peningkatan channel bertahap ($3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$)** mengikuti prinsip trade-off spasial-semantik, di mana penurunan resolusi spasial oleh `MaxPool` dikompensasi dengan penambahan kedalaman channel.
- **Kernel 3×3 dengan `padding=1`** dipilih karena terbukti lebih efisien dibanding kernel besar, sekaligus mempertahankan informasi pada tepi gambar yang mungkin mengandung bagian pesawat.
- **Global Average Pooling (GAP)** menggantikan *flatten* konvensional untuk menekan jumlah parameter secara drastis dari ~42 juta menjadi ~531 ribu, sekaligus meningkatkan invariansi translasi objek dalam frame.
- **Dropout 0,5** dipilih sebagai nilai regulasi yang agresif untuk mencegah *overfitting* pada model yang dilatih dari nol dengan data terbatas.

3.4. Skenario Model 2 (Pre-trained)

Pada skenario model kedua ini, arsitektur yang *pre-trained* `EfficientNetB2` dipanggil melalui library `torchvision.models` lengkap dengan bobot konvolusi asli yang dilatih melalui dataset *ImageNet-1k*. Proses penyesuaian adaptasi atau *fine-tuning* dirancang menggunakan teknik **Full**

Fine Tuning, artinya seluruh layer ekstraksi fitur yang ada di jaringan diputus dari penguncian gradien (*unfrozen layer*).



Gambar 3.5 Arsitektur Model EfficientNetB2

Secara arsitektural, lapisan klasifikasi bawaan model diganti dan dimodifikasi menjadi *classifier head* khusus sebagai berikut:

- **AdaptiveAvgPool2d & Flatten**
Melakukan perata rataan spasial fitur dan meratakan matriks multidimensi menjadi larik dimensi tunggal (tensor berukuran 1408 elemen)
- **Dropout Layer**
Diterapkan untuk menonaktifkan aktivasi neuron secara acak demi mengurangi dependensi yang terlalu kuat terhadap node fitur tertentu, sekaligus melengkapi upaya regularisasi.
- **Linear Layer**
Pemetaan koneksi Dense Layer secara penuh (fully connected) yang mendistribusikan 1.408 input menuju 20 output features yang merepresentasikan probabilitas prediksi terhadap 20 jenis klasifikasi pesawat.

3.4.1. Justifikasi Pemilihan Model

Beberapa kandidat arsitektur dipertimbangkan sebelum memilih EfficientNetB2:

- ResNet-50 memiliki ~25 juta parameter dengan resolusi input standar 224×224 , yang kurang optimal untuk menangkap detail halus antar varian pesawat.
- VGG-16 dengan ~138 juta parameter terlalu boros memori dan tidak efisien untuk GPU yang digunakan (RTX 3050 6GB).
- EfficientNetB0 terlalu ringan dengan resolusi input 224×224 , membatasi kemampuan menangkap perbedaan detail halus seperti proporsi fuselage atau bentuk winglet.
- EfficientNetB4 memiliki kapasitas lebih tinggi namun kebutuhan memorinya (~19 juta parameter, resolusi 380×380) terlalu besar untuk full fine-tuning pada GPU yang tersedia.

EfficientNetB2 dipilih karena menawarkan keseimbangan optimal: resolusi input 288×288 yang sesuai dengan pipeline preprocessing, ~7,7 juta parameter yang cukup ekspresif tanpa membebani GPU, serta metode Compound Scaling yang menyeimbangkan depth, width, dan resolution secara matematis untuk menghasilkan representasi fitur yang kaya.

Classifier head diganti dari 1000 kelas menjadi 20 kelas dengan satu linear layer langsung ($1408 \rightarrow 20$) tanpa hidden layer tambahan. Keputusan ini diambil karena penambahan hidden layer pada dataset kecil justru meningkatkan risiko overfitting. Dropout rate 0,3 dipilih lebih konservatif dibanding CNN dasar karena backbone pre-trained yang sudah stabil tidak memerlukan regularisasi yang terlalu agresif.

BAB IV

HASIL DAN PEMBAHASAN

Berikut adalah environment dari pengujian

Tabel 4.1 Environment Pengujian

Parameter	Value
Framework	PyTorch 2.2.2+cu121
CUDA	True
GPU	NVIDIA Geforce RTX 3050 6GB Laptop GPU

4.1. Analisis Proses Training

4.1.1. CNN Model

Model ini ditrain dengan konfigurasi berikut

Parameter	Value	Keterangan
Learning rate	1e-3	Nilai ini cukup besar, standar untuk melatih model dari awal (<i>from scratch</i>).
Epoch	200	Batas maksimal perulangan pelatihan model.
Patience	30	Batas toleransi epoch tanpa perbaikan <i>Validation Loss</i> sebelum <i>Early Stopping</i> diaktifkan.
Label smoothing	0.1	Mencegah model terlalu percaya diri pada tebakannya.
Optimizer	AdamW (weight_decay = 1e-2)	Digunakan untuk mengupdate bobot model. Sama dengan skenario Pre-trained, <i>weight_decay</i> diset 1e-2.
Scheduler	CosineAnnealingLR	Mengatur penurunan <i>learning rate</i> secara bertahap mengikuti kurva kosinus dengan batas bawah 1e-7.
Loss	CrossEntropyLoss	Fungsi <i>loss</i> standar untuk kasus klasifikasi <i>multi-class</i> .
AMP Scaler	True	Mengaktifkan <i>Automatic Mixed Precision</i> untuk mempercepat komputasi GPU.

Tabel 4.2 Konfigurasi Hyperparameter Pelatihan Custom CNN

Model ini menyelesaikan proses training hanya dalam 94 epoch dari batas maksimum 200 epoch yang ditetapkan. Fitur *early stopping* terpicu karena *validation loss* tidak menunjukkan perbaikan signifikan selama 30 epoch berturut-turut. Bobot model terbaik berhasil disimpan dan dikembalikan dari epoch ke-64, di mana *validation loss* mencapai nilai minimum sebesar 2,4559.

Tahap awal training (epoch 1-30) menunjukkan pola pembelajaran yang berfluktuasi. *Training loss* menurun perlahan, namun *validation loss* sangat tidak stabil dan melompat-lompat di kisaran angka 2,5 hingga 3,0. Pada epoch pertama, *training loss* tercatat sebesar 3,0440 dan *validation loss*

sebesar 2,9988. *Validation accuracy* pada fase ini masih sangat rendah, hanya berkisar 6% dan kemudian meningkat hingga 16% pada epoch ke-30.

Memasuki epoch 40 dan seterusnya, divergensi antara *training loss* dan *validation loss* mulai terlihat sangat jelas. *Training loss* terus menurun secara konsisten hingga menyentuh angka 1,9847 pada epoch terakhir (epoch 94). Sebaliknya, *validation loss* kembali fluktuatif dan stagnan di kisaran 2,4 - 2,8. *Training accuracy* pada akhir training mencapai 41,46%, sementara *validation accuracy* hanya mampu mencapai 20,69%.

Kesenjangan performa dan angka akurasi yang sangat rendah (sekitar 20%) ini mengindikasikan bahwa model gagal melakukan *learning* dan generalisasi dengan baik (mengalami *underfitting* dan mulai menunjukkan tanda-tanda *overfitting* ringan di akhir epoch).

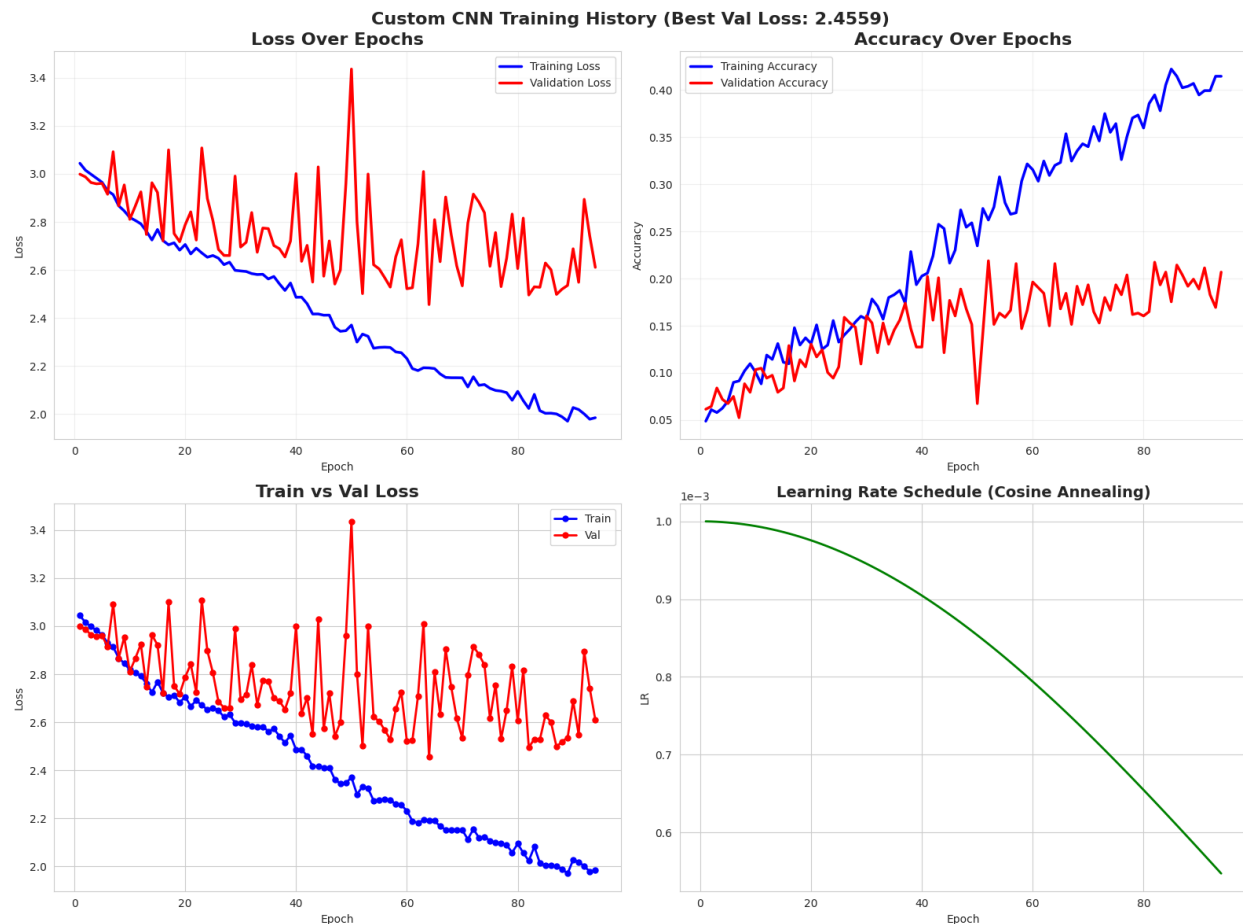
Pola kegagalan ini sangat relevan secara teori. Arsitektur Custom CNN yang dibangun sangat sederhana (hanya dengan 4 *convolutional block* dan ~531 ribu parameter) terbukti kekurangan kapasitas atau "kedalaman" (*depth*) untuk mengekstraksi fitur-fitur kompleks dan halus (*fine-grained*) dari gambar pesawat. Tanpa adanya inisialisasi bobot dasar (seperti pada *transfer learning*), model kesulitan untuk belajar dari awal, terutama karena dataset yang digunakan sangat kecil (hanya 667 gambar).

Ringkasan metrik performa model pada akhir training disajikan pada tabel berikut:

Tabel 4.3 Ringkasan Metrik Performa Terbaik Custom CNN (Epoch 64)

Metrik	Nilai
Epoch Terbaik (Best Val Loss)	64
Best Validation Loss	2,4559
Training Accuracy (Epoch 64)	32,01%
Validation Accuracy (Epoch 64)	21,59%
Training Loss (Epoch Terakhir / 94)	1,9847
Training Accuracy (Epoch Terakhir)	41,46%
Validation Accuracy (Epoch Terakhir)	20,69%
Epoch Early Stopping	94 dari 200

Penggunaan *learning rate scheduler* CosineAnnealingLR dirancang untuk menurunkan *learning rate* secara gradual dari nilai awal 1×10^{-3} . Penurunan yang halus ini bertujuan untuk membantu model berkonvergensi dengan lebih stabil. Namun, karena keterbatasan arsitektur dasar CNN itu sendiri, optimasi yang dilakukan tidak mampu memberikan dampak yang signifikan terhadap peningkatan akurasi pada data validasi. Dinamika metrik pelatihan ini dapat dilihat pada grafik di bawah ini :



Gambar 4.1 Grafik Riwayat Pelatihan (*Loss & Accuracy*) Custom CNN

4.1.2. EfficientNetB2

Model ini ditrain dengan konfigurasi berikut

Parameter	Value	Keterangan
Learning rate	2e-5	Nilai ini jauh lebih kecil dari Custom CNN, ideal untuk proses <i>fine-tuning</i> pada model <i>Pre-trained</i> .
Epoch	200	Batas maksimal perulangan pelatihan model.
Patience	30	Batas toleransi epoch tanpa perbaikan <i>Validation Loss</i> sebelum <i>Early Stopping</i> diaktifkan.
Label smoothing	0.1	Mencegah model terlalu percaya diri pada tebakannya.
Optimizer	AdamW (weight_decay = 1e-2)	Digunakan untuk mengupdate bobot model dengan pengaturan <i>weight_decay</i> 1e-2.
Scheduler	CosineAnnealingLR	Menurunkan <i>learning rate</i> secara halus dan periodik.
Loss	CrossEntropyLoss	Fungsi <i>loss</i> standar klasifikasi.
AMP Scaler	True	Mengaktifkan <i>Automatic Mixed Precision</i> untuk mempercepat komputasi GPU.

Tabel 4.4 Konfigurasi Hyperparameter Pelatihan EfficientNetB2

Model ini menyelesaikan proses training dalam 86 epoch sebelum *early stopping* dipicu, dari batas maksimum 200 epoch yang ditetapkan. *Early stopping* aktif karena validation loss tidak menunjukkan perbaikan selama 30 epoch berturut turut. Bobot model terbaik berhasil disimpan dan dikembalikan dari epoch ke-56, di mana validation loss mencapai nilai minimum sebesar 1,7456.

Tahap awal training (epoch 1–38) menunjukkan pola pembelajaran yang sehat dan konsisten, di mana baik training loss maupun validation loss turun secara bersamaan. Pada epoch 1, training loss sebesar 2,9987 dan validation loss sebesar 2,9872. Pada epoch 38, training loss telah turun menjadi 1,1717 sementara validation loss masih mengikuti di 1,7893. Validation accuracy pada

periode ini terus meningkat secara stabil dari 6,75% di epoch pertama menuju 49,63% di epoch 38.

Namun demikian, mulai epoch 39 dan seterusnya, mulai terlihat tanda-tanda divergensi yang mencirikan kondisi overfitting. Training loss terus menurun secara konsisten hingga mencapai 0,7255 pada epoch terakhir (epoch 86), sementara validation loss berhenti bergerak dan bahkan cenderung naik kembali di kisaran 1,75-1,85. Training accuracy pada akhir training mencapai 99,24%, sedangkan validation accuracy hanya stagnan di kisaran 51-53%. Kesenjangan yang sangat besar antara training accuracy (~99%) dan validation accuracy (~53%) merupakan indikator kuat terjadinya overfitting.

Pola overfitting ini dapat dijelaskan melalui beberapa faktor: Pertama, ukuran dataset training yang sangat kecil (hanya 667 gambar, atau rata-rata 33 gambar per kelas) tidak sebanding dengan kapasitas model yang memiliki 7,7 juta parameter yang seluruhnya trainable. Rasio parameter-per-sampel yang tinggi ini secara teori mempermudah model untuk menghafal data training. Kedua, meskipun augmentasi data telah diterapkan, keragaman visual yang dihasilkan tidak cukup untuk mensimulasikan variabilitas distribusi pada data validasi dan pengujian.

Ringkasan metrik performa model pada akhir training disajikan pada tabel berikut:

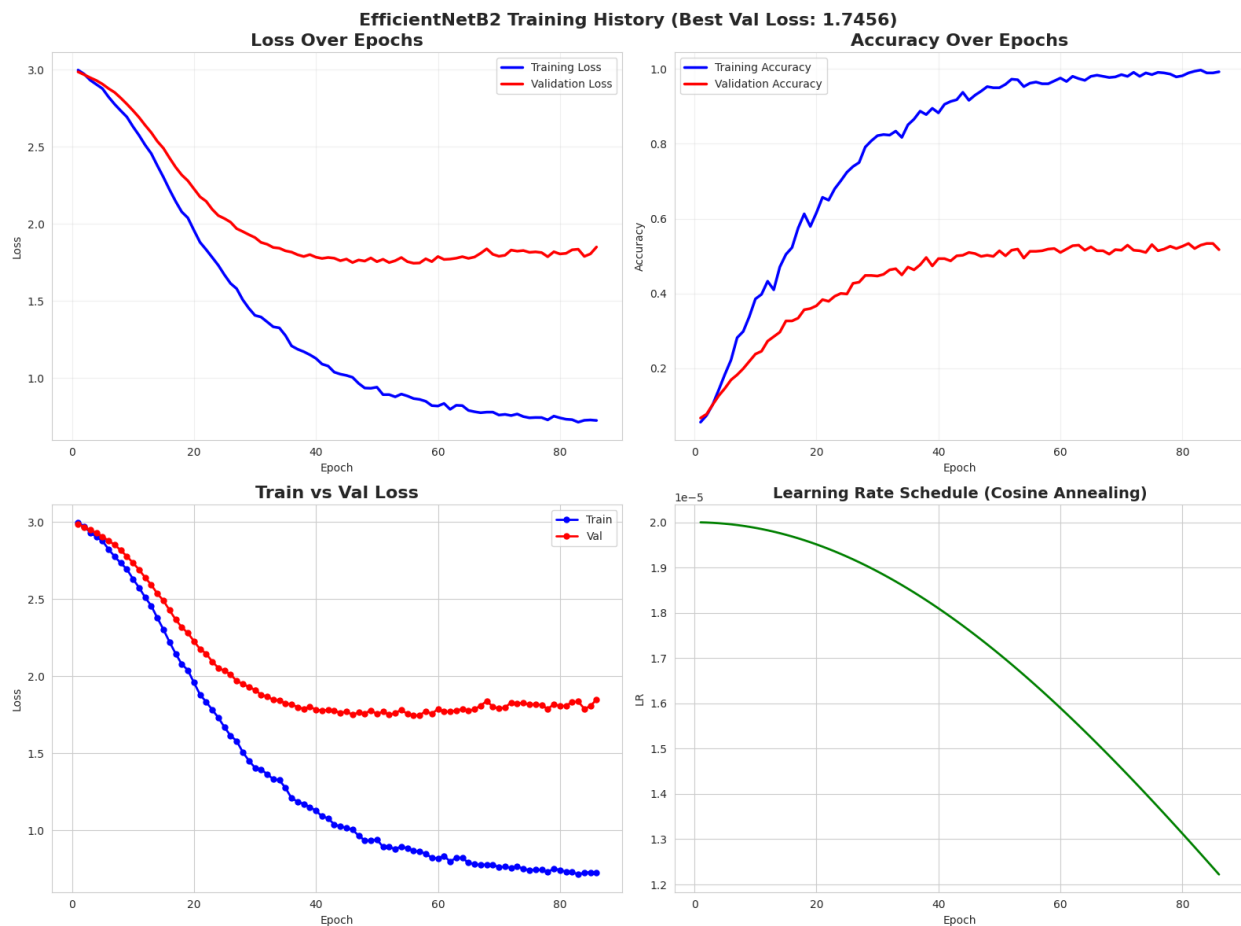
Tabel 4.5 Ringkasan Metrik Performa Terbaik EfficientNetB2 (Epoch 56)

Metrik	Nilai
Epoch Terbaik (Best Val Loss)	56
Best Validation Loss	1,7456
Training Accuracy (Epoch 56)	96,49%
Validation Accuracy (Epoch 56)	51,27%
Training Loss (Epoch Terakhir / 86)	0.7255
Training Accuracy (Epoch Terakhir)	99,24%
Validation Accuracy (Epoch Terakhir)	51,72%
Epoch Early Stopping	86 dari 200

Learning rate scheduler **CosineAnnealingLR** bekerja dengan menurunkan learning rate secara halus dari 2×10^{-5} pada epoch awal menuju $1,22 \times 10^{-5}$ di epoch 86. Penurunan yang gradual ini

bertujuan untuk membantu model berkonvergensi ke minimum lokal yang lebih tajam (*sharp minima*), namun pada kasus ini tidak cukup efektif untuk mengatasi kesenjangan generalisasi yang disebabkan oleh keterbatasan data.

Perlu dicatat bahwa konfigurasi yang digunakan dalam laporan ini merupakan hasil terbaik dari serangkaian eksperimen hyperparameter yang dilakukan sebelumnya. Berbagai konfigurasi yang telah dicoba mencakup variasi learning rate yang lebih tinggi (1×10^{-4} hingga 5×10^{-4}) yang menghasilkan konvergensi lebih cepat namun validation accuracy lebih rendah, strategi partial fine-tuning (hanya melatih classifier head dan beberapa layer akhir) yang menghasilkan underfitting, batch size yang lebih besar (32) yang memperparah overfitting, serta penggunaan dropout rate yang lebih tinggi (0,5) yang tidak signifikan memperbaiki generalisasi. Dari seluruh eksperimen tersebut, konfigurasi dengan learning rate 2×10^{-5} , full fine-tuning, dan dropout 0,3 menghasilkan test accuracy tertinggi sebesar 56,16%.



Gambar 4.2 Grafik Riwayat Pelatihan (Loss & Accuracy) EfficientNetB2

4.2. Perbandingan Metrik Performa

Tabel 4.6 Komparasi Metrik Performa: Custom CNN vs EfficientNetB2

Metrik	CNN Base Model	EfficientNetB2
Total Parameter	531.220	7.729.174
Epoch Terbaik / Total Epoch	64/94	56/86
Test Loss	2.5425	1.7125
Accuracy	0.1937	0,5616
Balanced Accuracy	0.1943	0.5627
Precision (Macro)	0.1672	0,5613
Precision (Micro)	0.1937	0,5616
Precision (Weighted)	0.1669	0,5611
Recall (Macro)	0.1943	0,5627
Recall (Micro)	0.1937	0,5616
Recall (Weighted)	0.1937	0,5616
F1 Score (Macro)	0.1715	0,5551
F1 Score (Micro)	0.1937	0,5616
F1 Score (Weighted)	0.1711	0,5545

Berdasarkan tabel komparasi di atas, terdapat beberapa analisis krusial terkait performa komputasi dan klasifikasi dari kedua model:

1. Analisis Akurasi dan Kemampuan Generalisasi

Perbedaan performa yang paling mencolok terlihat pada metrik *Accuracy*. Model EfficientNetB2 mengungguli Custom CNN dengan selisih yang cukup jauh, yaitu 56,16% berbanding 19,37%. Hal ini berbanding lurus dengan *Test Loss* EfficientNetB2 (1.7125) yang jauh lebih rendah dibandingkan CNN (2.5425). Angka ini mengonfirmasi bahwa pendekatan *Transfer Learning* menggunakan bobot *pre-trained* ImageNet memberikan fondasi ekstraksi fitur visual yang sangat kuat. EfficientNetB2 sudah mengenali pola geometris, tekstur, dan gradien warna, sehingga generalisasi pada data pesawat (FGVC) dapat dilakukan dengan jauh lebih baik meskipun jumlah *unseen data* pada *test set* cukup besar. Sebaliknya, Custom CNN yang dilatih dari awal (*from scratch*) dengan data terbatas gagal membentuk pola representasi yang kuat, sehingga berakhir pada kondisi *underfitting*.

2. Analisis Presisi, Recall, dan F1-Score

Dalam kasus FGVC di mana perbedaan antar kelas sangat halus (misal: Boeing 737-300 vs 737-400), metrik *Macro F1-Score* menjadi indikator keandalan utama karena memberikan bobot evaluasi yang setara untuk setiap kelas tanpa memedulikan mayoritas data.

- Custom CNN hanya mencetak *Macro F1-Score* sebesar 0.1715. Nilai *Precision* yang sejalan di angka 0.1672 menunjukkan bahwa saat model menebak sebuah kelas pesawat, tebakannya sebagian besar salah (*high false positive*).
- EfficientNetB2 mencetak *Macro F1-Score* sebesar 0.5551, seimbang dengan nilai *Balanced Accuracy*-nya (0.5627). Keseimbangan antara *Precision* dan *Recall* pada angka ~56% ini membuktikan bahwa EfficientNetB2 tidak hanya sekadar menebak kelas mayoritas secara acak, melainkan benar-benar membedakan fitur visual spesifik antar kelas dengan probabilitas ketepatan yang relatif stabil.

3. Analisis Kecepatan Konvergensi (Epoch)

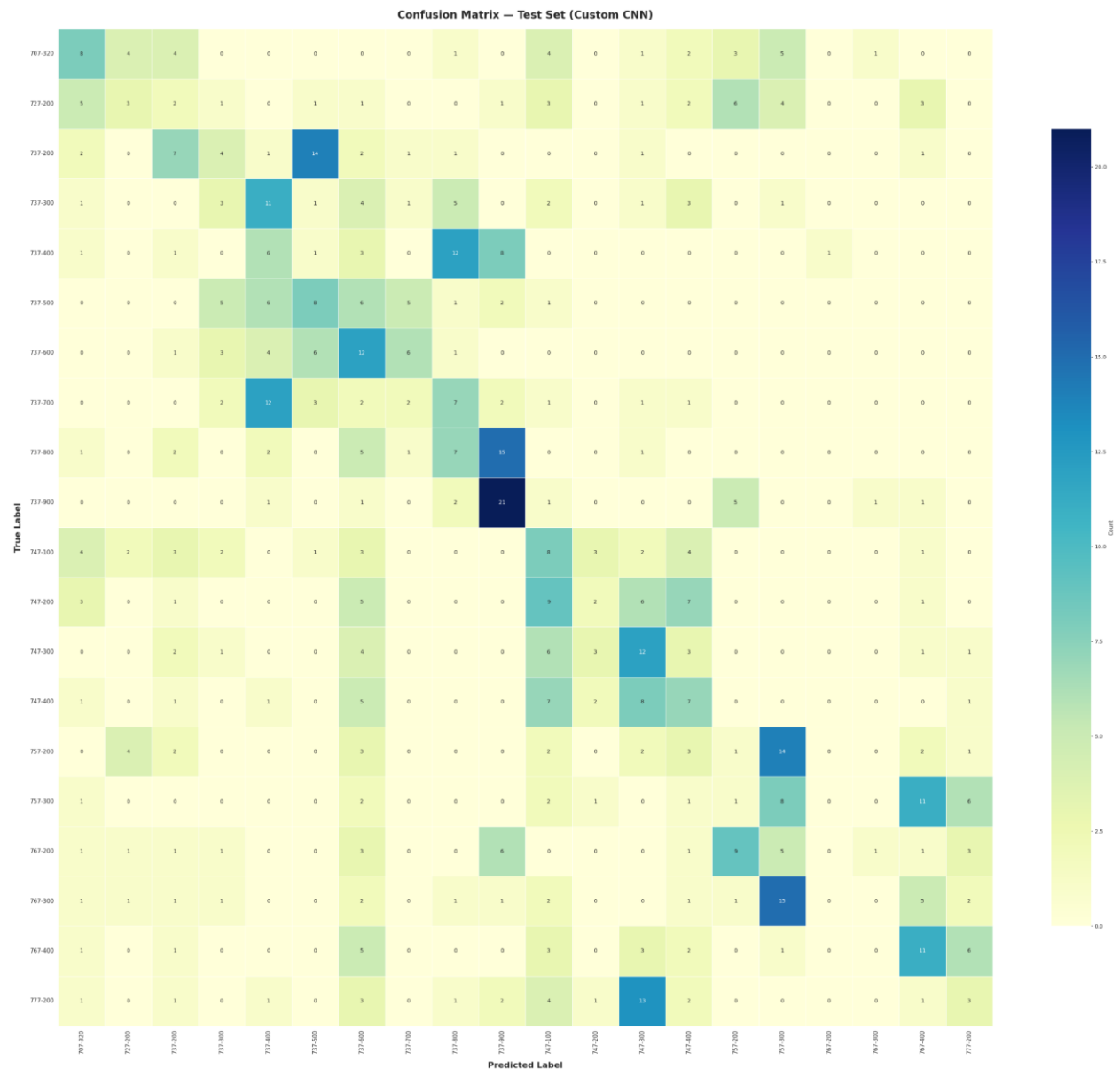
Efisiensi waktu pelatihan dapat dianalisis dari metrik *Epoch Terbaik*. EfficientNetB2 mencapai titik optimalnya (*Best Validation Loss*) lebih cepat, yaitu pada Epoch 56, dan memicu *Early Stopping* di Epoch 86. Di sisi lain, Custom CNN baru mencapai titik terbaiknya pada Epoch 64 dan berhenti pada Epoch 94. Fenomena ini membuktikan bahwa model *pre-trained* berkonvergensi lebih cepat ke titik minimum lokal karena tidak perlu membuang *epoch* awal untuk mempelajari fitur-fitur dasar gambar (seperti deteksi tepi atau garis) layaknya model yang dilatih dari nol.

4. Analisis Arsitektur dan Kompleksitas Komputasi

Keunggulan performa EfficientNetB2 tentunya datang dengan konsekuensi (*trade-off*) pada kompleksitas komputasi. EfficientNetB2 memiliki ~7,7 juta parameter, yang mana sekitar 14 kali lipat lebih besar dibandingkan Custom CNN yang hanya memiliki ~531 ribu parameter. Meskipun Custom CNN jauh lebih ringan dan cepat dalam hal waktu inferensi (*inference time*), arsitektur dasar ini terbukti kekurangan kapasitas kedalaman (*depth*) untuk menangani kerumitan tugas klasifikasi FGVC tingkat lanjut. Strategi *Compound Scaling* pada EfficientNetB2 terbukti sangat sepadan antara besarnya ukuran model dengan lonjakan performa akurasi yang dihasilkan.

4.3. Analisis Confusion Matrix

4.3.1. CNN Model



Gambar 4.3 Confusion Matrix Custom CNN pada Test Set

Visualisasi *confusion matrix* pada *test set* untuk model Custom CNN memperlihatkan distribusi prediksi yang sangat tersebar dan tidak terfokus. Garis diagonal utama (*true positives*) terlihat sangat redup dan terputus-putus, yang mengonfirmasi rendahnya akurasi klasifikasi secara keseluruhan. Model terlihat mengalami kebingungan (*confusion*) yang masif, tidak hanya antar varian pesawat dalam satu keluarga (*intra-class*), tetapi juga antar keluarga pesawat yang secara visual memiliki perbedaan bentuk yang lebih kentara (*inter-class*).

Dari matriks tersebut, hanya sedikit kelas yang berhasil dikenali dengan tingkat *recall* (sensitivitas) yang baik, seperti Boeing 737-900 (64%) dan 747-300 (36%). Sebaliknya, model sama sekali

gagal mengenali beberapa kelas tertentu. Hal ini terlihat pada pola tebakan di mana model cenderung "membuang" prediksi pada beberapa kelas mayoritas dan mengabaikan kelas lainnya.

Berikut adalah tabel rincian pasangan kelas yang paling sering salah diklasifikasikan oleh model Custom CNN:

Pasangan Kelas (True → Predicted)	Jumlah Kesalahan
767-300 → 757-300	15 kesalahan
737-800 → 737-900	15 kesalahan
757-200 → 757-300	14 kesalahan
737-200 → 737-500	14 kesalahan
777-200 → 747-300	13 kesalahan
737-700 → 737-400	12 kesalahan
737-400 → 737-800	12 kesalahan
757-300 → 767-400	11 kesalahan
737-300 → 737-400	11 kesalahan
747-200 → 747-100	9 kesalahan

Tabel 4.7 Pasangan Kelas dengan Kesalahan Prediksi Tertinggi (Custom CNN)

Pola kesalahan klasifikasi pada tabel di atas mencerminkan kelemahan fundamental dari arsitektur yang dibangun dari awal (*from scratch*) dengan data terbatas. Berbeda dengan model yang sudah memiliki pemahaman yang matang, Custom CNN ini gagal membentuk filter spasial yang memadai untuk mengenali bentuk sayap, badan pesawat (*fuselage*), atau jumlah mesin. Akibatnya, model sering kali salah memprediksi pesawat dengan proporsi dan desain yang jauh berbeda.

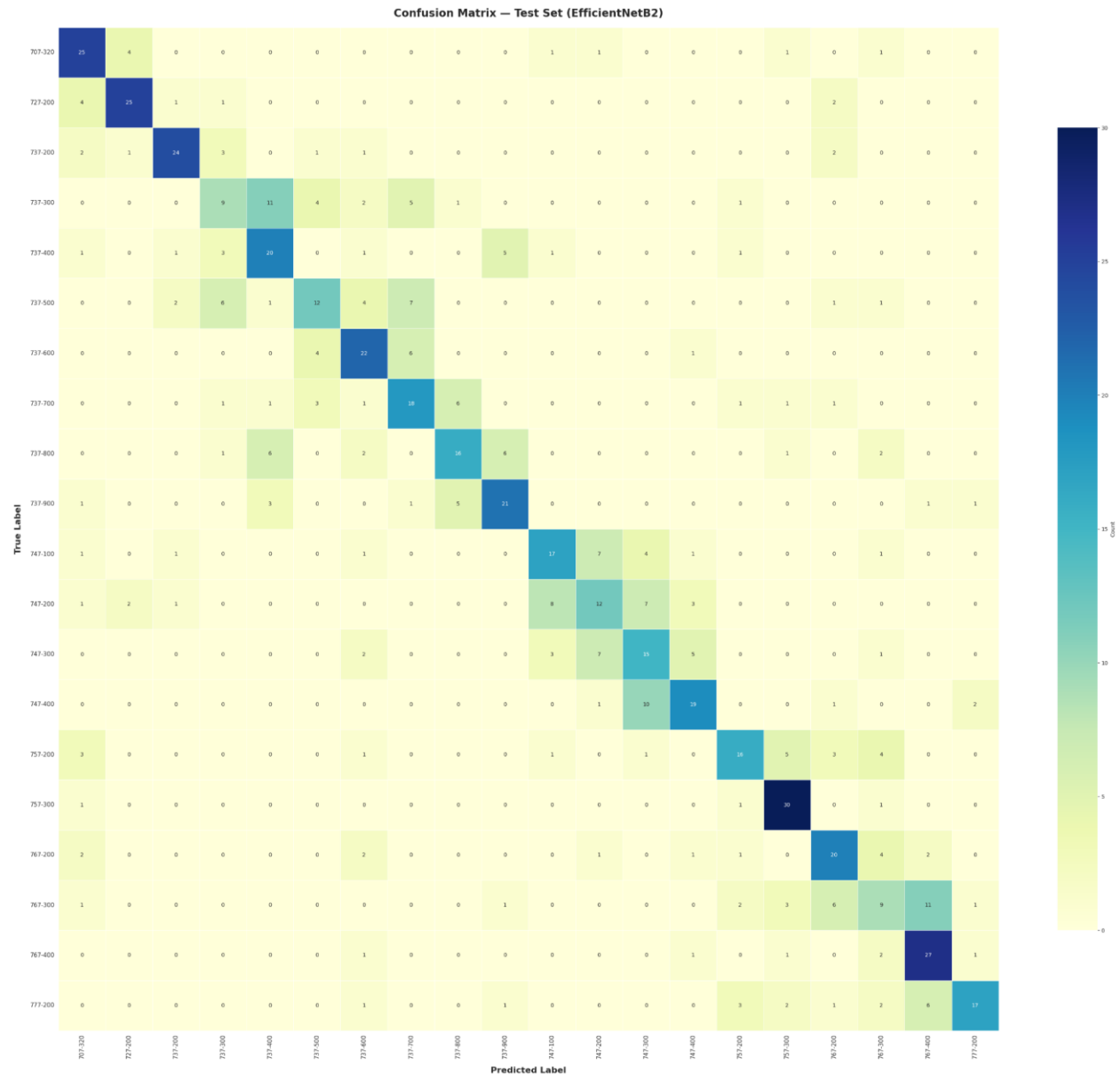
Kegagalan model dalam melakukan pemetaan fitur ini berujung pada nihilnya tebakan benar di beberapa kelas. Analisis per-kelas menunjukkan 10 kelas dengan performa terburuk sebagai berikut:

Kelas	Akurasi	Benar/Total
737-200	20,58%	7/34
737-400	18,18%	6/33
727-200	9,09%	3/33
737-300	9,09%	3/33
777-200	9,09%	3/33
737-700	6,06%	2/33
747-200	5,88%	2/34
757-200	2,94%	1/34
767-200	0%	0/33
767-300	0%	0/34

Tabel 4.8 Sepuluh Kelas dengan Performa Terburuk (Custom CNN)

Berdasarkan tabel di atas, kegagalan generalisasi model terlihat paling fatal pada kelas Boeing 767-200 dan Boeing 767-300, di mana model mencetak tingkat akurasi 0% (tidak ada satu pun gambar dari kedua kelas tersebut yang berhasil ditebak dengan benar). Tingkat akurasi yang berada di bawah 10% pada mayoritas kelas terbawah ini menjadi bukti bahwa CNN dasar dengan jumlah parameter ~531 ribu tidak memiliki kapasitas representasi (*representational capacity*) yang cukup untuk menyelesaikan studi kasus *Fine-Grained Visual Categorization* (FGVC).

4.3.2. EfficientNetB2



Gambar 4.4 Confusion Matrix EfficientNetB2 pada Test Set

Confusion matrix pada test set memperlihatkan pola kesalahan klasifikasi yang sangat informatif dan konsisten dengan sifat tugas fine-grained recognition pada dataset FGVC-Aircraft. Secara umum, sebagian besar kesalahan terjadi antara kelas-kelas yang memiliki kemiripan visual tinggi, khususnya antar varian dalam keluarga yang sama (misalnya sesama keluarga Boeing 737 atau sesama keluarga Boeing 747).

Diagonal utama confusion matrix menunjukkan bahwa beberapa kelas berhasil diklasifikasikan dengan cukup baik, antara lain 757-300 (akurasi per-kelas 90,91%), 707-320 (75,76%), dan 727-

200 (75,76%). Kelas-kelas ini kemungkinan memiliki karakteristik visual yang lebih distingtif atau lebih sedikit kemiripan dengan kelas lain. Sebaliknya, kelas-kelas dengan akurasi terendah adalah 767-300 (26,47%), 737-300 (27,27%), dan 747-200 (35,29%).

Pasangan Kelas (True → Predicted)	Jumlah Kesalahan
767-300 → 767-400	11 kesalahan
737-300 → 737-400	11 kesalahan
747-400 → 747-300	10 kesalahan
747-200 → 747-100	8 kesalahan
747-100 → 747-200	7 kesalahan
737-500 → 737-700	7 kesalahan
747-300 → 747-200	7 kesalahan
747-200 → 747-300	7 kesalahan
777-200 → 767-400	6 kesalahan
737-600 → 737-700	6 kesalahan

Tabel 4.9 Pasangan Kelas dengan Kesalahan Prediksi Tertinggi (EfficientNetB2)

Pola kesalahan ini sangat masuk akal secara domain: varian-varian dalam satu keluarga pesawat (misalnya 737-300 dengan 737-400, atau 747-100 dengan 747-200) memang memiliki perbedaan desain yang sangat halus, perbedaan utama sering kali hanya terletak pada panjang *fuselage*, posisi *wing-let*, atau detail mesin yang sulit terlihat jelas dari foto. Model visual berbasis CNN tanpa mekanisme attention eksplisit cenderung kesulitan membedakan perbedaan lokal yang sangat halus ini.

Analisis per-kelas menunjukkan 10 kelas dengan performa terburuk sebagai berikut:

Kelas	Akurasi	Benar/Total
737-700	54,55%	18/33
747-100	51,52%	17/33
777-200	51,52%	17/33
757-200	47,06%	16/34
737-800	47,06%	16/34
747-300	45,45%	15/33
747-200	35,29%	12/34
737-500	35,29%	12/34
737-300	27,27%	9/33
767-300	26,47%	9/34

Tabel 4.10 Sepuluh Kelas dengan Performa Terburuk (EfficientNetB2)

Berdasarkan tabel di atas, terlihat bahwa meskipun model EfficientNetB2 memiliki performa yang jauh lebih unggul daripada CNN dasar, model ini tetap menghadapi tantangan besar pada kelas-kelas dengan kemiripan visual yang sangat ekstrem, seperti keluarga Boeing 767 dan 737 varian lama. Namun, poin krusial yang membedakannya adalah nilai akurasi pada kelas terburuk sekalipun masih berada di angka 26%–35%, jauh di atas model Custom CNN yang mencatat angka 0% pada beberapa kelas. Hal ini menunjukkan bahwa *Transfer Learning* memungkinkan model untuk tetap menangkap fitur diskriminatif yang halus, meskipun belum sempurna dalam memisahkan varian pesawat yang memiliki perbedaan geometri sangat minimal.

4.4. Pembahasan

Berdasarkan keseluruhan hasil evaluasi dari kedua skenario model, eksperimen ini secara empiris membuktikan signifikansi pendekatan *Transfer Learning* dalam menyelesaikan tantangan *Fine-Grained Visual Categorization* (FGVC), terutama ketika dihadapkan pada keterbatasan jumlah data latih.

1. Keterbatasan Arsitektur Dasar (Training from Scratch)

Performa Custom CNN yang sangat rendah (akurasi ~19%) menyoroti fenomena *cold start* dalam *Deep Learning*. Melatih model dari awal (*from scratch*) untuk tugas *computer vision* yang kompleks membutuhkan dataset yang masif agar konvolusi pada *layer-layer* awal dapat belajar

mengenali fitur dasar seperti garis, tepi (*edges*), dan kurva, sebelum akhirnya menyusun fitur-fitur tersebut menjadi representasi objek yang utuh. Dengan arsitektur yang relatif dangkal (~531 ribu parameter) dan ukuran *training set* yang kecil, Custom CNN terbukti tidak memiliki ruang kapasitas (*capacity*) maupun variasi data yang cukup untuk membentuk representasi fitur hierarkis tersebut. Hal ini berujung pada kegagalan ekstraksi fitur spasial dan menyebabkan model tidak mampu membedakan satu jenis pesawat dengan pesawat lainnya.

2. Efektivitas Transfer Learning dan Compound Scaling

Sebaliknya, model EfficientNetB2 mampu mencetak akurasi yang jauh lebih superior (~56%) berkat pemanfaatan bobot *pre-trained* dari dataset ImageNet. Lapisan-lapisan ekstraksi fitur pada EfficientNetB2 sudah matang dan mengenali pola geometris, gradien warna, serta tekstur secara universal. Proses *fine-tuning* yang dilakukan hanya berfungsi untuk menggeser (*align*) pengetahuan dasar tersebut agar lebih peka terhadap fitur spesifik pesawat terbang. Selain itu, teknik *Compound Scaling* bawaan EfficientNet yang menyeimbangkan antara kedalaman (*depth*), lebar (*width*), dan resolusi (*resolution*) layer terbukti sangat efektif dalam memaksimalkan performa klasifikasi tanpa membuat beban komputasi menjadi tidak masuk akal.

3. Tantangan Inheren pada FGVC

Meskipun *Transfer Learning* memberikan lonjakan performa yang masif, hasil analisis matriks kebingungan (*confusion matrix*) pada EfficientNetB2 menunjukkan bahwa tugas klasifikasi FGVC tetap menjadi tantangan yang sangat berat. Kelas-kelas yang berada dalam satu varian manufaktur yang sama (misalnya Boeing 737 varian 200, 300, hingga 900) memiliki tingkat kemiripan visual (*inter-class similarity*) yang sangat ekstrem. Perbedaan antar kelas tersebut sering kali hanya terletak pada detail mikroskopis seperti rasio panjang *fuselage*, bentuk lekukan *winglet*, atau proporsi *engine nacelle*, yang terkadang terdistorsi oleh sudut pengambilan gambar (*viewpoint*), pencahayaan, atau latar belakang (*background noise*).

Oleh karena itu, dapat disimpulkan bahwa arsitektur CNN modern seperti EfficientNetB2 sudah sangat mumpuni sebagai *feature extractor*. Namun, untuk mencapai akurasi FGVC yang mendekati sempurna, model mungkin memerlukan mekanisme tambahan yang difokuskan pada lokalisasi bagian-bagian spesifik objek (*part-based model* atau *attention mechanism*) agar model dipaksa untuk fokus secara eksklusif pada area-area diskriminatif seperti hidung, sayap, atau ekor pesawat.

BAB V

PENUTUP

5.1. Kesimpulan

Berdasarkan hasil eksperimen dan analisis yang telah dilakukan pada praktikum klasifikasi citra pesawat (*Fine-Grained Aircraft Recognition*), dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Implementasi Model:** Praktikum ini berhasil mengimplementasikan dua skenario pemodelan *Computer Vision*, yaitu *Custom CNN Model* yang dibangun dari awal (*from scratch*) dan *EfficientNetB2 Model* yang menggunakan pendekatan *Transfer Learning* dengan bobot *pre-trained* ImageNet. Keduanya berhasil dijalankan menggunakan *pipeline* yang seragam, mulai dari *preprocessing* hingga evaluasi akhir.
2. **Efektivitas Transfer Learning:** Pendekatan *Transfer Learning* terbukti menjadi solusi paling efektif dalam menangani masalah keterbatasan data. Model yang sudah memiliki pemahaman fitur visual dari dataset skala besar (ImageNet) mampu berkonvergensi lebih cepat dan memberikan generalisasi yang lebih baik dibandingkan model yang harus mempelajari seluruh pola visual dari nol.
3. **Perbandingan Performa:** Terdapat perbedaan performa yang sangat signifikan antara kedua model. Model *EfficientNetB2* mencetak akurasi sebesar 56,16%, jauh mengungguli *Custom CNN* yang hanya mencapai akurasi 19,37%. Hal ini menunjukkan bahwa untuk tugas klasifikasi yang mendalam (*fine-grained*) dengan dataset terbatas (667 gambar), arsitektur dasar CNN saja tidak cukup untuk mengekstraksi fitur visual yang memadai.

5.2. Saran

Guna pengembangan lebih lanjut untuk meningkatkan akurasi klasifikasi pada domain *Fine-Grained Visual Categorization* (FGVC), disarankan beberapa hal sebagai berikut:

1. **Augmentasi dan Penambahan Data:** Mengingat dataset yang digunakan tergolong kecil, penggunaan teknik augmentasi data yang lebih variatif (seperti *CutMix* atau *Mixup*) atau penambahan jumlah sampel data per kelas sangat disarankan untuk membantu model mempelajari variasi sudut pandang pesawat yang lebih luas.
2. **Eksperimen dengan Mekanisme Attention:** Untuk membedakan varian pesawat yang memiliki perbedaan sangat halus pada bagian tertentu, penggunaan arsitektur yang memiliki mekanisme *Attention* (seperti *Vision Transformer* atau *Convolutional Block Attention Module*) disarankan agar model dapat fokus pada bagian-bagian diskriminatif seperti bentuk mesin, moncong, atau *winglet*.
3. **Hyperparameter Tuning yang Lebih Luas:** Melakukan optimasi lebih lanjut terhadap *learning rate*, penggunaan *optimizer* yang berbeda, serta pengaturan jumlah *convolutional*

layer pada model dasar diharapkan dapat menekan angka *loss* dan meningkatkan stabilitas pelatihan model.

4. CNN base model mungkin belum cocok untuk tugas ini, vision transformer mungkin akan cocok karena kemampuan generalisasi yang lebih baik